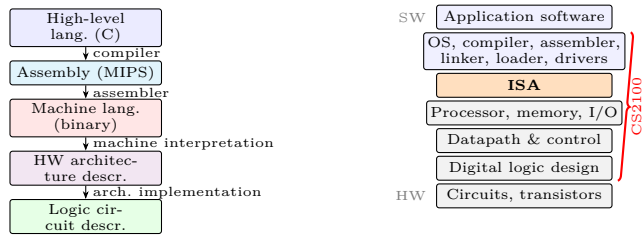


CS2100 CHEATSHEET | Part A: C programming (L1, L2, L4, L5)

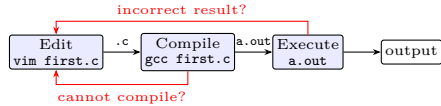
L1 The big picture: levels of representation



Topics: C → data representation → assembly (MIPS) → datapath & control → pipelining, cache → combinational & sequential circuits. Tools: C debugger, QTSpm (MIPS simulator), Logisim. Textbooks: DLD (Aaron Tan), COD 4e (Patterson & Hennessy). Final exam is open book (printed/written), calculators allowed.

L2a C basics: program, memory, types

C: 1972, Dennis Ritchie, Bell Labs, for UNIX.

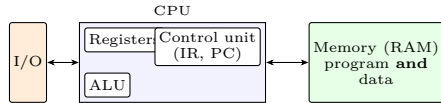


Flags: -o name (exe name, default a.out), -Wall (all warnings, e.g. “used uninitialized”), -lm (link math lib), -DDEBUG/-UDEBUG (define/undefine macro).

```
// Converts distance in miles to kilometres.
#include <stdio.h> /* printf, scanf definitions */
#define KMS_PER_MILE 1.609 /* conversion constant */
int main(int argc, char *argv[]) {
    float miles, kms; // declarations
    printf("Enter distance in miles: "); // input
    scanf("%f", &miles);
    kms = KMS_PER_MILE * miles; // compute
    printf("That equals %9.2f km.\n", kms); // output
    return 0;
}
```

General form: preprocessor directives; global declarations; functions {declarations; executable statements (input, compute, output)}. ; ends a statement, {} is a block (Python uses indentation). Comments: /* */ (only this is ANSI C) and //. Parts of a program: preprocessor directives, reserved words, special symbols, variables, functions, comments.

von Neumann architecture (a.k.a. Princeton / stored-program):



! Uninitialised variables hold garbage, **not** 0. Before scanf: miles=?, kms=?; after input miles=10.5; after compute kms=16.89.

Variables have a name (identifier), a type, a value and an *address*. int count = 3; declares and initialises. Names: letters, digits, _; must not start with a digit; no reserved words; case-sensitive; avoid standard identifiers (printf). Valid: maxEntries, _X123, this_IS_a_long_name. Invalid: 1Letter, double, return, joe's, ice cream, T*S. Redundant init: int c=0; c=123;.

Data types (sunfire, via sizeof):

type	bytes	notes	fmt
int	4	-2^{31} ($-2,147,483,648$) to $2^{31} - 1$	%d
float	4	real; $1.5e-2=0.015$, $1.2E+5=120000.0$	%f
double	8	real (scanf needs %lf)	%lf
char	1	'A', '\n', single quotes	%c

Variants: **short**, **long**. Strongly typed (declare type: C, Java, Pascal; Java/-

Pascal stronger since C allows implicit conversions and pointer casts) vs weakly typed (JS var).

Program structure : preprocessing directives; global declarations (struct, enum, typedef, extern); global data definitions; global function definitions (prototype = return type, name, params; body in braces: local declarations/definitions, blocks, statements, expressions). **extern** int x; = declaration vs int x; = definition. **extern** int f(int a); declaration vs int f(int a){...} definition. *Parameter* = in definition (a); *argument* = in call (f(3)). .h files hold declarations, .c files definitions.

Preprocessor : (1) header inclusion #include <stdio.h> (I/O), <math.h> (compile with -lm), <string.h>, <ctype.h>; (2) macro expansion #define PI 3.142 (textual replace before compiling, ALL CAPS); (3) conditional compilation #if defined(DEBUG) ... #endif.

I/O (variadic functions): printf(format, list), scanf(format, &var). age = value, &age = address of the cell (varies per run).

spec	type	used in
%c	char	printf/scanf
%d	int	printf/scanf
%f	float or double	printf
%f	float	scanf
%lf	double	scanf
%e	float/double	printf (scientific)
%p	pointer (hex)	printf
%s	string	printf/scanf

%5d: width 5, right-justified. %8.3f: width 8, 3 d.p. scanf: no width/precision. Escapes: \n newline, \t tab, \" quote, %% percent.

L2b Compute: assignment, operators, types

Function body = declaration statements (what memory cells) + executable statements. **main** is where execution starts.

Assignment = means “assign right to left”, *not* equality. Left side is the **lvalue** (must be assignable): 32 = a; and a + b = c; give “lvalue required” errors. Cascade is right-assoc: a = b = c = 3+6; ≡ a=(b=(c=9)). **Side effect**: an assignment returns the value assigned: z = a = 12; a = 5 + (b = 10); gives b=10, a=15 (legal but avoid).

Arithmetic binary + - * / % are left-assoc: 46/15/2 = 3/2 = 1; 19%7%3 = 5%3 = 2. Unary + - right-assoc (x = -23; p = +4*10 gives 40). Order: parentheses, then precedence, then associativity; * / % above + -. Truncation on store: int n = 9*0.5; gives 4.

Operator precedence (high to low):

type	operators	assoc
primary	() [] . -> x++ x-	L→R
unary	* & + - ! ~ ++x -x (cast) sizeof	R→L
multiplicative	* / %	L→R
additive	+ -	L→R
relational	< > <= >=	L→R
equality	== !=	L→R
logical and	&&	L→R
logical or		L→R
ternary	?:	R→L
assignment	= += -= *= /= %=	R→L

Mixed types & casts : int m=10/4→2; float p=10/4→2.0 (int division first!); float q=10/4.0→2.5; int n=10/4.0→2; int r=-10/4.0→-2 (truncates toward 0). Cast (type) expr: with int aa=6; float ff=15.8; (float)aa/4=1.5; (int)ff/aa=15/6=2; (float)(aa/4)=1.0.

! C % is *remainder* (sign follows dividend): -10%4 = -2. Python % is *modulo*: -10%4 = 2, and Python -10//4 = -3 (floor). C / on ints truncates toward 0.

L2c Selection & repetition

if / else; switch (switch expression must be discrete type; falls through until break; default if no case matches; stack case 'A': case 'B': to share code; GCC extension case 65 ... 67:).

Conditions : < <= > >= !=. No chained 1<x<=5 like Python. **No Boolean type in ANSI C**: 0 = false, any non-zero = true, 1 printed for true: int a=(2>3), b=(3>2); gives a=0, b=1. Logical: && and, || or, ! not. Ternary: y = (x>0) ? x : -x;

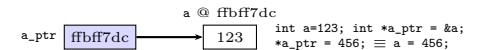
Example a=4,b=-2,c=0: x = (a>b || b>c && a==b) = 1 (&& binds tighter: T || (F&&F)); y = ((a>b || b>c) && a==b) = 0; z = ((a>b) && !(b>c)) = 1. Add parentheses!

Short-circuit : e1 || e2 skips e2 if e1 true; e1 && e2 skips e2 if e1 false. So if ((a!=0) && (b/a > 3)) is safe when a=0.

Loops : while (c) {}; do {} while (c); (body runs ≥ 1 time, note the ;); for (init; cond; update) {}; for (int i=0;i<N;i++) runs N times; **for**(;;) infinite. **break** exits / **continue** skips to next iteration of the **innermost** enclosing loop only. E.g. for i=1..5 {print i; if(i==3) break; print "Ya"} prints 1 Ya 2 Ya 3; with continue: 1 Ya 2 Ya 3 4 Ya 5 Ya. Nested (i=1..3, j=1..5, break at j==3): each i prints i,1 Ya i,2 Ya i,3 then next i.

L4a/b Pointers

C is “low” among HLLs: pointers (direct memory manipulation) and bitwise operators.



- ▶ & = address-of; print with %p (hex, changes run to run).
 - ▶ Declare **type *name**; (type = type of pointee; suffix _ptr/_p). Only addresses can be assigned: a_ptr = &a;
 - ▶ * = indirection/dereference: *a_ptr is a synonym for a.
 - ▶ double a, *b; reads as “*b is a double”, so b is a pointer to double. double *b = &a; OK; double b = &a; illegal.
 - ▶ With b=&a; *b=12.34;: printf("%f",a) and ("%f",*b) print 12.340000; ("%f",b) compiles with warning; ("%f",*a) is an error; print pointer with %p.
- Example #1** : int i=10,j=20,*p; p=&i; *p=*p+2; (i=12) p=&j; *p=i; (j=12).

Trace : a=8,b=15,c=23; p1=&b; p2=&c; p3=p2; print 1: 15 23 23; *p1 *= a; (b=120); while(*p2>0){*p2-=a; (*p1)++;} (c: 23,15,7,-1; b: 123). Prints 2: 123 -1 -1 and 3: 8 123 -1.

Pointer arithmetic : p++ advances by sizeof(*p): int +4, float +4, char +1, double +8; ap += 3 adds 12 for int.

! int *n; *n = 123; n points nowhere (garbage address) ⇒ Segmentation fault (core dumped). Delete the big **core** file.

Why pointers : let a function change caller’s variables / “return” several values; pass an array (address of first element).

L4c/d Functions, pass-by-value, scope

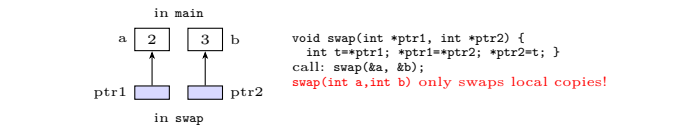
Library : #include <math.h> + gcc -lm. abs(int)→int; the rest take/return double: ceil, floor, fabs, exp, log, log10, sqrt, pow(x,y), sin, cos, tan (trig in radians). Prototype e.g. double pow(double x, double y).

User-defined : put **prototypes** (return type, name, param types; names optional) before main, definitions after. Without prototype the compiler assumes implicit int return ⇒ warning “implicit declaration” then error “conflicting types”. void function returns nothing.

Washer example (rim area = $\pi(d_2/2)^2 - \pi(d_1/2)^2$, volume = rim area × thickness):

```
double circle_area(double); // prototype
... rim_area = circle_area(d2) - circle_area(d1);
double circle_area(double diameter) { return PI * pow(diameter/2, 2); }
```

Pass-by-value : actual params (arguments) are *copied* into formal params. Params and locals are local to the function (**scope rule**); an **activation record** is pushed on the call stack at call and popped at return, so they are **automatic** variables; **static** variables persist after the function returns. A function cannot use another function’s locals (return a + x; in f where a is main’s: error).



Examples : `f(int x,y,z){x=3+y; y=10*x; z=x+y+z;}` with `a=9,b=-2,c=5`: prints `x=1,y=10,z=16` but `a,b,c` unchanged. Pointer version `f(&a,&b,&c)` using `*x,*y,*z`: `a=1,b=10,c=16`. Printing `x` with `%d` warns (it is an address); use `%p`.

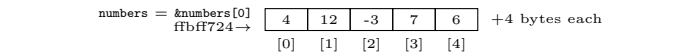
Scoping : one global scope; local scopes nest; every `{}` makes a new scope; **lexical** scoping (decided at compile time from program text). Inner declaration shadows outer.

```
int x = -1; // global
void foo() { printf("5. x = %d\n", x); } // sees global: -1
int main(void) { int x; x = 10; // 1. x = 10
    if (x > 0) { int x; x = 20; } // 2. x = 20 (inner x)
    /* 3. x = 10 */ x = 30; /* 4. x = 30 */ foo(); /* 6. x = 30 */ }
```

`static` is a storage-class specifier: on a global it limits it to **file scope**; on a local it changes **duration** (keeps value across calls). (`const` is another declaration keyword.)

L5a Arrays

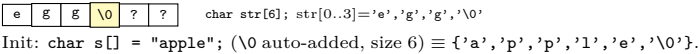
Homogeneous collection, **contiguous** memory, index from 0. `int c[30];` (type, name, size).



- Init: `int a[3]={5,9,10}; int b[]={1,2,3};` (size 3); `int c[5]={17,3,10};` rest are 0. `int e[2]={1,2,3};` warning (excess); `f[5]={...};` after declaration is a compile error.
- Array name in an expression = address of first element (`a == &a[0]`); `&a[1]` is 4 bytes later. It is a **constant pointer**: `dest = source`; is **illegal**; copy with a loop (or `memcpy`, out of scope).
- Parameter `int arr[]` $\equiv$ `int *arr`; any size inside `[ ]` is ignored, so pass size separately: `int sumArray(int [], int)`;
- Function receives the address $\Rightarrow$ it **can modify** the caller's array (no `&` needed). Use `const float arr[]` to promise not to modify.

L5b Strings

String = char array terminated by `'\0'` (ASCII 0). Without it, it is just a char array.



`I/O` : `fgets(str, size, stdin)` reads up to `size-1` chars or until newline, **and keeps** the `\n`: strip with `len=strlen(str); if(str[len-1]=='\n') str[len-1]='\0';`. `scanf("%s",str)` stops at whitespace (input "My book" gives "My"); `fgets` gives "My book"). `puts(str)` prints + newline; `printf("%s")`. Avoid `gets` (unsafe).

`<string.h>` : `strlen(s)` (chars before `\0`); `strcmp(s1,s2)` compares ASCII: `< 0` if `s1` lexicographically smaller, `> 0` if greater, 0 if equal; `strncpy(s1,s2,n)`; `strcpy(s1,s2)` copies `s2` into `s1`, returns `s1` (overflows if `s2` too long!); `strncpy(s1,s2,n).name = "Matthew"`; does **not** work. `<ctype.h>`: `toupper(c)` etc. Remove-vowels: loop, `switch(toupper(str[i]))` {case 'A': ... case 'U': break; default: `newstr[count++] = str[i];`} then `newstr[count]='\0';`

! Missing `\0`: `printf("%s")/strlen` run past the array until a stray 0, e.g. "Length = 8, apple??<" or illegal memory access.

L5c/d Structures

Heterogeneous members (fields). A **type** is not a variable (no memory allocated).

```
typedef struct { int day, month, year; } date_t;
typedef struct { int cardNum; date_t expiryDate; } card_t; // nested
typedef struct { int stuNum; float score; char grade; } result_t; // <- ; !
card_t card1 = {888888, {31, 12, 2020}};
```

```
result_t r1 = {123321, 93.5, 'A'}, r2;
r2.stuNum = 456654; card1.expiryDate.year = 2021; // dot operator
scanf("%d %f %c", &r1.stuNum, &r1.score, &r1.grade);
r2 = r1; // struct assignment OK (member-wise copy), unlike arrays
struct ACC { int acctNum; float balance; }; // struct tag version
struct ACC acct1, acct_arr[10];
```

Type defined by `struct` itself; `typedef` only names it (no new type, no variable); tag optional (anonymous `struct {int i; float b;}; x`; legal); repeated equivalent typedefs allowed (warning only). Define types after preprocessor directives, before prototypes.

Functions : structs are **passed/returned by value** (whole struct copied); `result_t max_and_average(int,int,int)` returns a struct into the caller. A function receiving `player_t player` changes only its copy ("Brusco, 23" stays). To modify, pass the address:

```
void change_name_and_age(player_t *p) {
    strcpy(p->name, "Alexandra"); // same as (*p).name
    p->age = 25; } // call: change_name_and_age(&player1);
```

! `*p.name` means `*(p.name)` since `.` binds tighter than `*`. Write `(*p).name` or `p->name`.

Arrays of structs (e.g. `store_t storeList[]` with name, (x,y), radius) are passed as pointers, so functions *can* modify elements.

L3 Number Representation | L9 More on Numbers

L3a Bits, bases, conversions

Same bits, different meaning: `01000110` = 70 as int, 'F' as char. `0xC0D00000` = `-1060110336` as int, `-6.5` as float. Byte = 8 bits, nibble = 4, word = multiple of bytes (arch-dependent). N bits $\rightarrow 2^N$ values; M values need $\lceil \log_2 M \rceil$ bits (32 values: 5 bits; 1000 values: 10 bits).

Weighted-positional : $(a_n \dots a_0.f_1 \dots f_m)_R = \sum a_i R^i + \sum f_j R^{-j}$. Octal digits 0 to 7, hex 0 to F. Notation: C 032 = octal, 0x32 = hex; QTSpm 0x100; Verilog 8'b11110000 = 8'hF0 = 8'd240.

Base-R $\rightarrow$ decimal : multiply by weights. $1101.101_2 = 8+4+1+0.5+0.125 = 13.625$; $572.6_8 = 378.75$; $2A.8_{16} = 42.5$; $341.24_5 = 96.56$.

Decimal $\rightarrow$ base R : whole part by **repeated division** by R (first remainder = LSB); fraction by **repeated multiplication** by R (first carry = MSB), until fraction is 0 or enough digits. $43 \div 2$: rem 1 (LSB), 1,0,1,0,1 (MSB) $\left| \begin{array}{l} 0.3125 \times 2 = 0.625 \rightarrow 0 \text{ (MSB)} \\ 0.625 \rightarrow 1, 0.25 \rightarrow 0, 0.5 \rightarrow 1 \text{ (LSB)} \\ 0.3125_{10} = .0101_2 \end{array} \right.$
 $43_{10} = 101011_2$

Between arbitrary bases go via decimal. **Shortcuts** for 2/8/16: group bits from the binary point outwards, 3 for octal, 4 for hex: $(10111011001.101110)_2 = (2731.56)_8$; $(11110011001.10111000)_2 = (5D9.B8)_{16}$.

L3b ASCII & negative numbers

ASCII : 7 bits (+1 parity bit, odd or even). `'0'`=48 (0x30), `'A'`=65 (0x41), `'a'`=97 (0x61), `' '`=32, `'\0'`=0, `'\n'`=10. Row = 3 MSBs, column = 4 LSBs: 000/001 control chars, 010 space and punctuation, 011 digits, 100/101 uppercase, 110/111 lowercase. Integers 0 to 127 and chars are interchangeable in C: `printf("%c",65)  $\rightarrow$  A, printf("%d", 'F')  $\rightarrow$  70.`

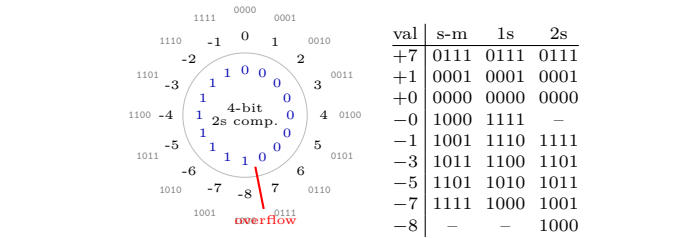
Past-year Q : int `n=2147483640`; add 1 ten times. After 7 adds $n = 2^{31} - 1$; 8th wraps to `-2147483648`; final `-2147483646` (32-bit 2s complement overflow).

n-bit	negate	range	zeros
Sign-mag	flip sign bit	$-(2^{n-1} - 1)$ to $2^{n-1} - 1$	+0, -0
1s comp.	invert all bits; $-x = 2^n - x - 1$	$-(2^{n-1} - 1)$ to $2^{n-1} - 1$	0..0, 1..1
2s comp.	invert all, add 1; $-x = 2^n - x$	-2^{n-1} to $2^{n-1} - 1$	one zero
Excess-K	value = code - K	code 0 is -K	one

MSB is the sign (0 +, 1 -) in all three signed schemes. 8-bit: s-m and 1s: ± 127 ; 2s: -128 to 127.

E.g. 8-bit: -12 : 1s 11110011, 2s 11110100. -14 : 1s 11110001, 2s 11110010. -80 : 80 = 01010000, 1s 10101111, 2s 10110000. s-m: `00110100`=+52, `10010011`=-19.

Intuition: 2s complement is a clock face: 1111 is one step "before" 0000, so it is -1. The MSB has weight -2^{n-1} : $1011 = -8 + 2 + 1 = -5$. Adding just goes clockwise; overflow is crossing the $+7/-8$ seam.



Fractions : complement works too. 1s: $-0101.01 = 1010.10$, $-111000.101 = 000111.010$; 2s: $-0101.01 = 1010.11$ (add 1 at the last bit). Complement idea: 9's complement in base 10 ($X + \bar{X} = 9999$).

L3c Add/Sub, overflow, excess

Terms: augend + addend = sum; minuend - subtrahend = difference; multiplier $\times$ multiplier = product; dividend $\div$ divisor = quotient, remainder. + and $\times$ commute; - and $\div$ don't.

2s complement $A+B$: add; **ignore carry out** of MSB; **overflow** iff carry-in $\neq$ carry-out of MSB, i.e. `pos+pos=neg` or `neg+neg=pos`. $A - B = A + (-B)$ (take 2s comp of B, add).

1s complement $A+B$: add; if carry out of MSB, **add 1 (end-around carry)**; overflow iff result sign differs from (same) signs of A and B.

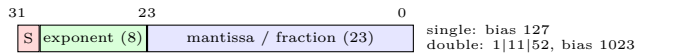
4-bit 2s		4-bit 1s	
3+4: 0011+0100=0111 ok		3+4 = 0111 ok	
-2+-6: 1110+1010=1000 ok,-8		-2+-5: 1101+1010=1011 $\rightarrow$ 1000 ok	
6+-3: 0110+1101=1001 ok		5+-5: 0101+1010=1111 ok (-0)	
4+-7: 0100+1001=1101 ok,-3		-3+-7: 1100+1000=10100 $\rightarrow$ 0101 OVF	
-3+-6: 1101+1010=1011 OVF			
5+6: 0101+0110=1011 OVF			
6-1: 0110+1111=10101 ok,5			
-5-4: 1011+1100=10111 OVF			

Excess-K (biased): code = value + K; spreads range evenly by translation. 3-bit excess-4: `000`=-4 ... `100`=0 ... `111`=3. 4-bit excess-8: `0000`=-8, `1000`=0, `1111`=7 (excess-7 also possible: -7 to 8). Codes compare like unsigned.

L3d Real numbers: fixed & floating point

Fixed point : fixed split of integer/fraction bits (range vs precision trade-off). 8-bit, 6.2 format, 2s: `011010.11` = 26.75; `111110.11` = -000001.01 = -1.25.

IEEE 754 (base 2): **sign | exponent (biased) | mantissa**; normalised with **implicit leading 1** (not stored).



$v = (-1)^S \times 1.\text{mantissa} \times 2^{E-127}$. $110.1_2 = 1.101 \times 2^2$ (store 101); $0.00101101_2 = 1.01101 \times 2^{-3}$ (store 01101).

Example $-6.5 = -110.1_2 = -1.101 \times 2^2$; $E = 2 + 127 = 129 = 10000001$: 1 10000001 010000000000000000000000 = `0xC0D00000`.

Recipe : sign; convert to binary; normalise to $1.f \times 2^e$; store `e`+bias; store `f` left-aligned, pad 0s. Reverse to decode.

L9 More on numbers

Everything in a computer is a finite number $\Rightarrow$ approximation $\Rightarrow$ errors that accumulate. Normalisation gives at most one bit pattern per real value (no duplicates).

Range : set of distinct representable values; of interest: largest/smallest positive and negative (non-zero). **Overflow**: beyond largest magnitude. **Underflow**: non-zero value smaller in magnitude than smallest representable (no such issue for integers).

Precision : min non-zero gap between two values: integers 1; fixed point 2^{-k} (k fraction bits); floating point is *relative* (depends on exponent). FP32: next number after 1.0×2^e is $(1 + 2^{-23}) \times 2^e$; about 7 to 8 decimal digits.

Rounding : pick a representable result. IEEE requires computing with ≥ 3 extra bits: **guard**, **round**, **sticky** (OR of all bits beyond round). Extra bits arise naturally (n-bit $\times$ n-bit gives 2n bits; adding FP needs shifting one operand to align binary points). Modes: **round to nearest (default)**, tie: pick the one whose LSB is 0 (“to even”); toward zero; toward $+\infty$; toward $-\infty$.

! FP arithmetic is **not associative**: $(A \text{ op } B) \text{ op } C = \text{round}(\text{round}(A \text{ op } B) \text{ op } C)$. Summing a list largest-first vs smallest-first generally differs. Errors propagate in long computations.

Patriot missile (1991) : time in tenths of a second $\times 0.1$ stored in 24-bit fixed point; 0.1 not exact in binary; after 100 h error ≈ 0.34 s; Scud not intercepted, 28 killed.

Special values (exponent all 0s or all 1s):

exponent	fraction	value
all 0	all 0	± 0
all 0	non-zero	denormalised: $(-1)^S \times 0.f \times 2^{1-\text{bias}}$ (hidden bit 0, exponent fixed at 1−bias; tiny numbers)
other	any	normalised $1.f \times 2^{E-\text{bias}}$
all 1	all 0	$\pm \infty$
all 1	non-zero	NaN (error)

AI-era formats (LLMs have $\sim 10^{12}$ parameters; low per-weight accuracy is OK, so **quantise**):

format	bits	exp	frac	main use
FP64	64	11	52	scientific
FP32	32	8	23	general
BF16	16	8	7	AI training (FP32 range)
FP16	16	5	10	graphics, AI
FP8 E4M3	8	4	3	weights, activations
FP8 E5M2	8	5	2	gradients
FP4 E2M1	4	2	1	experimental

Block-scaling variants: MXFP8/4, NVFP4, HiF8/4 (multi-level scale factors).

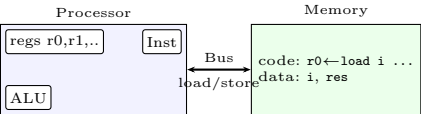
Integer quantisation: scale = $(2^{b-1} - 1) / \max |x|$; INT = round($x \times \text{scale}$). E.g. INT8, max $|x| = 12.47$: scale = $127/12.47 = 10.18$; $-12.47 \rightarrow -127$, $5.72 \rightarrow 58$, $8.31 \rightarrow 85$. INT4 less accurate than FP4, but FP4 needs HW support.

L6 to L8 MIPS | L10 Instruction Set Architecture

L6 ISA, assembly, registers

ISA = abstraction/interface between hardware and low-level software: everything a programmer needs to make machine code work. Many implementations (cost/performance) run the same software (x86/IA-32: 80386 (1985) to Pentium 4 (2005), AMD, Transmeta).

C A+B $\xrightarrow{\text{compiler}}$ add A,B $\xrightarrow{\text{assembler}}$ 1000 1100 1010 0000. Assembly is human-readable symbolic machine code; may have **pseudo-instructions** (syntactic sugar, translated to ≥ 1 real instructions, e.g. move $\$rd, \$rs \rightarrow \text{add } \$rd, \$rs, \$zero$); only real instructions count for performance.



Walkthrough (for($i=1; i<10; i++$) res+=i;): memory is slow, so keep values in **registers**: $r0 \leftarrow \text{load } i$; $r1 \leftarrow \text{load res}$; $r1 \leftarrow r1 + r0$; $r0 \leftarrow r0 + 1$; if $r0 < 10$ repeat; $i \leftarrow \text{store } r0$. Concepts: **stored-program** (code+data in memory); **load-store**

model; 3 instruction kinds: memory, calculation, control flow. Typical: load, compute, store.

Registers : fast, few (16 to 32, limited by encoding), **no data type** (instruction decides interpretation). MIPS has 32:

name	#	use	name	#	use
\$zero	0	const 0	\$t8-\$t9	24-25	more temps
\$at	1	assembler	\$k0-\$k1	26-27	OS kernel
\$v0-\$v1	2-3	results	\$gp	28	global ptr
\$a0-\$a3	4-7	arguments	\$sp	29	stack ptr
\$t0-\$t7	8-15	temporaries	\$fp	30	frame ptr
\$s0-\$s7	16-23	program vars	\$ra	31	return addr

(Preserved across call: $\$s, \$gp, \$sp, \$fp, \$ra$.) Other register kinds: special-purpose, FP, flag, vector, control, debug.

L6 Arithmetic & logical instructions

Syntax op dest, src1, src2; one instruction per line; # comment. Arithmetic is **register-to-register**. Max 2 sources per instruction, so split: $f = (g+h) - (i+j) \rightarrow \text{add } \$t0, \$s1, \$s2$; $\text{add } \$t1, \$s3, \$s4$; $\text{sub } \$s0, \$t0, \$t1$. Order matters for sub $\$s0, \$s1, \$s2$ ($\$s1 - \$s2$).

Immediates : addi $\$s0, \$s0, 4$; constant is 16-bit 2s complement $[-2^{15}, 2^{15} - 1]$, **sign-extended**. No subi: use negative addi. \$zero gives $f=g$: add $\$s0, \$s1, \$zero$ (pseudo move).

Logical (register = 32 raw bits): sll/srl (C $\ll/\gg$; fill with 0; shift $n = \times / \div 2^n$, e.g. $a*8 \rightarrow \text{sll } \$s0, \$s0, 3$); and/andi (**masking**: 1 keeps bit, 0 clears; last 12 bits: andi $\$t0, \$t1, 0xFFF$); or/ori (force bits to 1); nor; xor/xori. C $\gg$ is arithmetic in GCC/Clang.

No NOT: nor $\$t0, \$t0, \$zero$ or xor with all-1s (keep ISA small). No nori (little need) but xori exists.

! Logical immediates (andi, ori, xori) are **zero-extended** (raw bit pattern C16); arithmetic ones (addi, slti, offsets) are **sign-extended** (C16_{2s}). ori $\$t0, \$t1, 0xFFFF$: upper 16 bits stay 0.

Large constant 0xAAAAFOFO: lui $\$t0, 0xAAAA$ (upper 16, lower zeroed) then ori $\$t0, \$t0, 0xFOFO$.

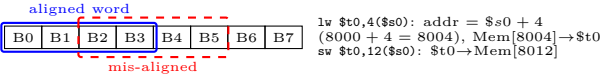
instr	meaning	instr	meaning
add rd,rs,rt	rd=rs+rt	addi rt,rs,C16 _{2s}	rt=rs+C
sub rd,rs,rt	rd=rs−rt	sll rd,rt,C5	rd=rt $\ll$ C5
and rd,rs,rt	rd=rs&rt	srl rd,rt,C5	rd=rt $\gg$ C5
or rd,rs,rt	rd=rs rt	andi rt,rs,C16	rt=rs&C16
nor rd,rs,rt	rd=rs rt	ori rt,rs,C16	rt=rs C16
xor rd,rs,rt	rd=rs^rt	xori rt,rs,C16	rt=rs^C16

C5 $\in [0, 31]$ (shifting by 32 clears the register). Why learn C for MIPS: C controls memory, so C maps closely to MIPS.

L7 Memory, load/store

Memory = big 1-D array of bytes (**byte addressing**); k -bit address $\Rightarrow 2^k$ locations. **Word** = usually 2^2 bytes, the unit of transfer, often = register/instruction size. MIPS: 32 regs $\times$ 32 bits, 32-bit words and addresses, Mem[0], Mem[4], ..., Mem[4294967292].

Word-aligned: address is a multiple of the word size (4): lowest 2 bits = 00.



- Only load/store touch memory; arithmetic operands must be registers.
- A[7] = h + A[10]; (base of A in \$s3, h in \$s2): lw $\$t0, 40(\$s3)$; add $\$t0, \$s2, \$t0$; sw $\$t0, 28(\$s3)$ (offset = index $\times$ 4).
- lb/sb (byte; offset need not be a multiple of 4), lh/sh, lw/lwr/swl/swr. lw/sw require aligned address; pseudo ulw/usw for unaligned.

! Consecutive words differ by 4, not 1. Registers have no type: in lw $\$t2, 0(\$t0)$, $\$t0$ holds an address; in add it holds a value.

Swap v[k], v[k+1] (k in \$5, base in \$4): sll $\$2, \$5, 2$; add $\$2, \$4, \$2$; lw $\$15, 0(\$2)$; lw $\$16, 4(\$2)$; sw $\$16, 0(\$2)$; sw $\$15, 4(\$2)$.

L7 Branches, jumps, loops

beq $\$r1, \$r2, L$ (if equal goto L), bne $\$r1, \$r2, L$, j L (always; $\equiv$ beq $\$s0, \$s0, L$). Labels are anchors, not instructions. Selection = branch down; repetition = branch up. **Trick**: invert the condition to skip the body.

```
# if (i==j) f=g+h; else f=g-h;
bne $s3,$s4,Else
add $s0,$s1,$s2
j Exit
Else: sub $s0,$s1,$s2
Exit:

# while (j==k) i=i+1;
Loop: bne $s4,$s5,Exit
      addi $s3,$s3,1
      j Loop
Exit:

# for (i=0; i<10; i++) a=a+5; i:$s0 a:$s2
add $s0,$zero,$zero
addi $s1,$zero,10
Loop: beq $s0,$s1,Exit
      addi $s2,$s2,5
      addi $s0,$s0,1
      j Loop
Exit:
```

Decompile: beq $\$s1, \$s2, \text{Exit}$; add $\$s0, \$zero, \$zero$; Exit: is if ($i \neq j$) $f = 0$;

Inequalities : slt $\$t0, \$s1, \$s2$ sets $\$t0 = (\$s1 < \$s2) ? 1 : 0$; also slti. Pseudo blt $\$s1, \$s2, L = \text{slt } \$t0, \$s1, \$s2$; bne $\$t0, \$zero, L$. Others: $a > b \Leftrightarrow b < a$; $a \geq b \Leftrightarrow (a < b)$; $a \leq b \Leftrightarrow (b < a)$.

Array + loop : init counters/pointers; Label: compute address, load, work; update; compare & branch. Count zeros in 40-word A ($\$t0 = \&A$, result $\$t8$):

```
# v1: index
addi $t8,$zero,0
addi $t1,$zero,0
addi $t2,$zero,40
loop: bge $t1,$t2,end
sll $t3,$t1,2 # i*4
add $t4,$t0,$t3 # &A[i]
lw $t5,0($t4)
bne $t5,$zero,skip
addi $t8,$t8,1
skip: addi $t1,$t1,1
j loop
end:

# v2: pointer (faster)
addi $t8,$zero,0
addi $t1,$t0,0 # ptr
addi $t2,$t0,160 # &A[40]
loop: bge $t1,$t2,end
lw $t3,0($t1)
bne $t3,$zero,skip
addi $t8,$t8,1
skip: addi $t1,$t1,4
j loop
end:
```

Exercises : counting executed instructions: count setup once, loop body $\times$ iterations, and remember the last (failing) branch. Ex3 ($\$t1$ becomes 19 so beq $\$t1, \$zero, \text{Loop}$ is not taken; no looping): 6 instr, $\$t2=20$. Ex4 (bne $\$t2, \$t0$ with $\$t2=4$): $3+4 \times 3=15$ instr, $\$t1=0+1+2+3=6$. Ex5 (ulw loop from $\$t0+10$ down to $\$t0+1$): bne executed 10, taken 9, $2+4 \times 10=42$ instr, bytes read = offsets 1..13 $\Rightarrow 13$ unique bytes.

Covered: arithmetic, large constants, load/store, branch/jump, arrays, syscalls (labs). Not covered: procedures, linker/loader, stack frames, recursion, interrupts.

L8 Instruction encoding (R, I, J)

All instructions are **32 bits**; few, regular formats. **opcode** always bits 31:26; rs, rt at the same place in R and I.

	31	26	21	16	11	6	0
R	opcode		rs	rt	rd	shamt	funct
I	opcode		rs	rt	immediate (16)		
J	opcode		target address (26)				

R-format (add, sub, and, or, nor, xor, slt, sll, srl): opcode = 0; **funct** picks the op; rs = 1st src, rt = 2nd src, rd = dest; shamt = shift amount (0 if not a shift). Shifts: rs = 0, source in **rt**.

I-format (addi, andi, ori, slti, lw, sw, beq, bne): opcode alone identifies instr; rs = source; **rt = destination** (for arith/load). Immediate is signed except andi/ori/xori. lw rt, C(rs). Branch beq rs, rt, label: first reg is **rs**.

Examples (fields in decimal, then hex):

```
add $8,$9,$10      010101010132 0x012A4020
                   000000 01001 01010 01000 00000 100000
sll $8,$9,4         010101010132 0x0009A100
                   010101010132 0x00E55020
add $10,$7,$5       017151010132 0x00E55020
addi $21,$22,-50    81221211-50 0x22B5FFCE
                   001000 10110 10101 111111111001110
lw $9,12($8)        35181912 0x8D09000C
beq $9,$0,End       4191013 0x11200003
```

Opcode / funct (green card) :

R (op 0)	funct	I	op	I/J	op
sll	0x00	addi	0x08	lw	0x23
sr1	0x02	slti	0x0A	sw	0x2B
jr	0x08	andi	0x0C	lb	0x20
add	0x20 (32)	ori	0x0D	sb	0x28
sub	0x22 (34)	xori	0x0E	beq	0x04
and	0x24 (36)	lui	0x0F	bne	0x05
or	0x25 (37)			j	0x02
xor/nor	0x26/0x27			jal	0x03
slt	0x2A (42)				

Green card: SignExtImm = {16{imm[15]}, imm}; ZeroExtImm = {16{0}, imm}; BranchAddr = {14{imm[15]}, imm, 00}; JumpAddr = {PC+4[31:28], address, 00}. Also jr \$ra (jump to register), jal L (\$ra = PC+4, jump).

L8 Branch & jump addressing

PC holds the address of the current instruction; instructions are word-aligned.

Branch (PC-relative) : loops are small, so encode an offset. Immediate counts words (instructions) from PC+4:

not taken: $PC \leftarrow PC + 4$; taken: $PC \leftarrow (PC + 4) + \text{imm} \times 4$

Range $\pm 2^{15}$ words = $\pm 2^{17}$ bytes. Imm = (#instructions between the branch and the target, counting from the one after the branch). E.g. beq at 0, End at 16: $\text{imm} = (16 - 4)/4 = 3$. beq \$0,\$0,Loop at 12 back to 0: $\text{imm} = (0 - 16)/4 = -4$.

Jump (pseudo-direct) : 26-bit field; append 00 (word aligned) $\rightarrow$ 28 bits; take top 4 bits from PC+4:

PC+4[31:28]	26-bit target from instruction	00	= jump target within 256MB block
-------------	--------------------------------	----	----------------------------------

Cannot jump across a 256MB boundary (0x0..., 0x1..., 0x2...); use jr (address in register). Example: j Loop with Loop at 8: target field = $8/4 = 2$.

Far branch : if L1 is out of range of beq \$s0,\$s1,L1, use bne \$s0,\$s1,Skip; j L1; Skip:.

Addressing modes (MIPS) :

- **Register**: operand in register (add, sub, and, or, xor, nor, slt, sll, srl).
- **Immediate**: constant in instruction (addi, andi, ori, xori, slti).
- **Base/displacement**: Mem[reg + const] (lw, sw).
- **PC-relative**: (PC+4) + const $\times$ 4 (beq, bne).
- **Pseudo-direct**: PC[31:28] : 26 bits : 00 (j).

Branches and load/store are both I-format but branches use PC-relative, load/store use base addressing. Shifts are R-format; other immediate instr are I-format.

L10 ISA design: 5 concepts

CISC (x86-32/IA32, VAX: polynomial-multiply instruction): complex instructions, small program size (memory was precious), complex HW, little room for optimisation.

RISC (MIPS, ARM): small simple set, easy to build/optmise HW; software combines simple ops.

#1 Data storage : where are operands and results? (von Neumann: data in memory.)

Stack	Accumulator	Reg (load-store)	Mem-Mem
Push A	Load A	Load R1,A	Add C,A,B
Push B	Add B	Load R2,B	
Add	Store C	Add R3,R1,R2	
Pop C		Store R3,C	

Stack: operands implicit on top of stack. Accumulator: one operand implicit

in the accumulator (IBM 701, DEC PDP-8). GPR: only explicit operands; register-memory (Motorola 68000, Intel 80386) or register-register/load-store (MIPS, DEC Alpha). Memory-memory: all in memory (DEC VAX). **GPR dominates**: CPUs much faster than memory and registers plentiful; multicore needs parallelism/less memory reliance; better compilers. RISC: reg-reg; CISC: mix (IA32, for compatibility).

#2 Memory & addressing : k -bit address bus $\Rightarrow 2^k$ locations; n -bit data bus; MAR/MDR; control lines (R/W).

Endianness (byte order within a word), e.g. 0xDEADBEEF:

addr +0	+1	+2	+3	Big: MSB at lowest address (IBM 360/370, 68000, MIPS (SGI), SPARC). Little: LSB at lowest (x86, VAX, Alpha). MIPS is implementation specific; the online MIPS interpreter is little-endian.
Big	DE	AD	BE	EF
Little	EF	BE	AD	DE

Addressing modes (MIPS has only register, immediate, displacement):

mode	example	meaning
Register	Add R4,R3	$R4 \leftarrow R4 + R3$
Immediate	Add R4,#3	$R4 \leftarrow R4 + 3$
Displacement	Add R4,100(R1)	$R4 \leftarrow R4 + M[100 + R1]$
Register indirect	Add R4,(R1)	$R4 \leftarrow R4 + M[R1]$
Indexed/base	Add R3,(R1+R2)	$R3 \leftarrow R3 + M[R1 + R2]$
Direct/absolute	Add R1,(1001)	$R1 \leftarrow R1 + M[1001]$
Memory indirect	Add R1,(R3)	$R1 \leftarrow R1 + M[M[R3]]$
Auto-increment	Add R1,(R2)+	$R1 \leftarrow R1 + M[R2]; R2 \leftarrow R2 + d$
Auto-decrement	Add R1,-(R2)	$R2 \leftarrow R2 - d; R1 \leftarrow R1 + M[R2]$
Scaled	Add R1,100(R2)[R3]	$R1 \leftarrow R1 + M[100 + R2 + R3 \times d]$

#3 Operations : data movement (load, store, mem-mem/reg-reg move, I/O, push/pop), arithmetic (int/FP add/sub/mul/div), shift (incl. rotate), logical (not/and/or/set/clear), control flow (jump, branch), subroutine (call/return), interrupt (trap/return), synchronisation (test&set, atomic r-m-w), string (search/move/compare), graphics (pixel/vertex, (de)compression).

Frequency: load 22%, cond. branch 20%, compare 16%, store 12%, add 8%, AND 6%, sub 5%, move reg-reg 4%, call 1%, return 1% (96%). **Make the common case fast** (Amdahl): halve time of top 4 (70%) and slow the rest by 50%: $0.35t + 0.45t = 0.80t$, still a win.

#4 Instruction formats : **variable length** (x86: 1 to 17 bytes; VAX: 1 to 54) is compact/flexible but needs multi-step fetch/decode; **fixed length** (MIPS, PowerPC: 4 bytes) easy fetch/decode, simpler pipelining, but bits are scarce; **hybrid**. Instruction = opcode + 0 or more operands; the op designates operand type/size: char 8, half-word 16, word 32, single FP 1 word, double FP 2 words. A new 32-bit ISA should support 8/16/32-bit ints and 32/64-bit FP (64-bit ISA: 64-bit ints too).

#5 Encoding : trade-offs between code size, speed, complexity; decide #registers, #addressing modes, #operands. Forces: many regs and modes vs small code vs easy (fixed) length. Choices: variable (VAX, x86), fixed (Alpha, ARM, MIPS, PowerPC, SPARC, SuperH), hybrid (IBM 360/70, MIPS16, Thumb, TI TMS320C54x).

Expanding opcode: opcode length varies by instruction type (MIPS funct is an example); design the most constrained type (needs most operand bits) first.

Intuition: Expanding opcode is like prefix-free phone numbers: reserve some short opcodes as “escape” prefixes, then the freed operand bits extend the opcode for instructions that need fewer operands. Count: if k opcode patterns are used by type A, the remaining $2^n - k$ patterns each expand by the extra bits for type B.

To learn any ISA: data storage, addressing modes, operations, formats, encoding (+ assembler conventions; procedures/stack and privileged instructions not covered).

C $\rightarrow$ MIPS compilation patterns (L7 advice: learn these)

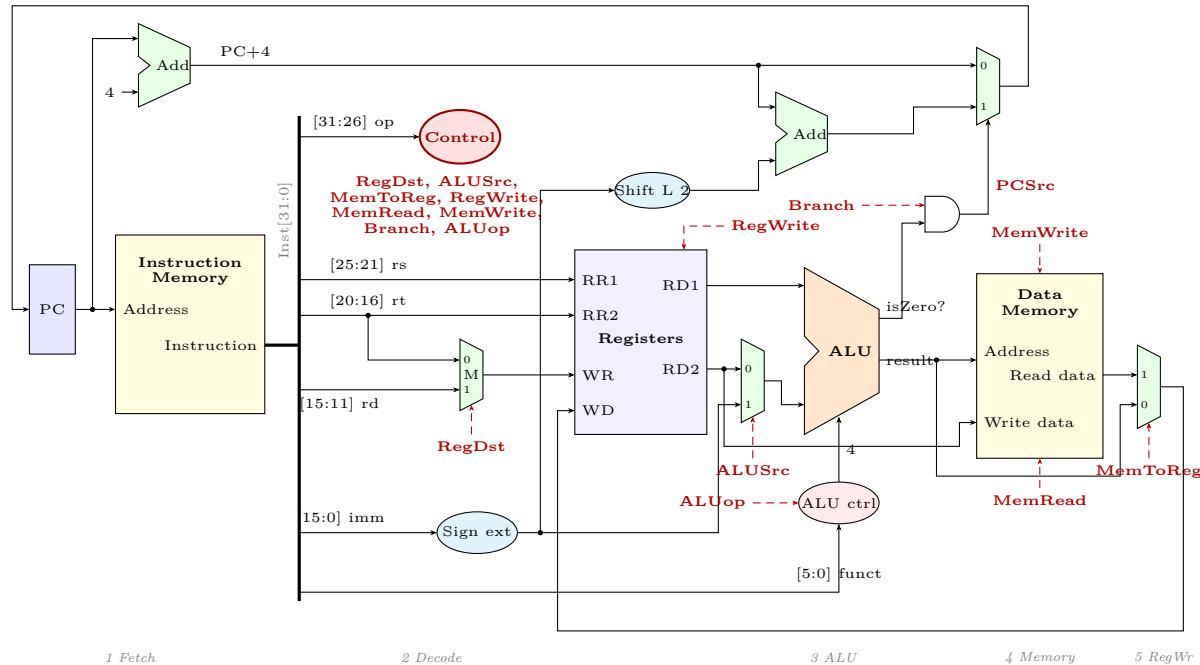
Rewrite the construct with if (...) goto, invert the condition, then map $==/!=$ to beq/bne, $<$ to slt+**bne**/beq.

<pre># do { body } while (a != b); Loop: <body> bne \$a,\$b,Loop # if (a < b) X; slt \$t0,\$a,\$b beq \$t0,\$zero,Skip <X> Skip: for(init;cond;upd) body $\equiv$ init; while(cond){body; upd;}. Array A[i] (words): sll \$t,\$i,2; add \$t,\$base,\$t; lw \$x,0(\$t). Constant index k: lw \$x,4k(\$base). Bytes (char): no scaling, use lb/sb. Instruction counting: total = setup + (iterations $\times$ body) + final exit check.</pre>	<pre># while (a < b) body; Loop: slt \$t0,\$a,\$b beq \$t0,\$zero,Exit <body> j Loop Exit: # if (a >= b) X; == !(a<b) slt \$t0,\$a,\$b bne \$t0,\$zero,Skip</pre>
--	--

Quick reference: formulas

n -bit unsigned	0 to $2^n - 1$
n -bit 2s comp	-2^{n-1} to $2^{n-1} - 1$
n -bit s-m / 1s	$-(2^{n-1} - 1)$ to $2^{n-1} - 1$
1s negate	$2^n - x - 1$ (invert)
2s negate	$2^n - x$ (invert, +1)
IEEE single	$(-1)^S 1.f \times 2^{E-127}$
IEEE double	bias 1023, $1+11+52$
branch target	$(PC + 4) + \text{imm} \times 4$
jump target	$PC+4[31:28] \parallel \text{addr26} \parallel 00$
imm range	addi: $[-2^{15}, 2^{15} - 1]$; shamt $[0, 31]$
lw/sw offset	index $\times 4$ (words)

L11 Datapath | L12 Control | single-cycle MIPS (supports add, sub, and, or, slt, lw, sw, beq; +addi, bne in principle)



Control signals (L12)

signal	stage	0 (false)	1 (true)
RegDst	Decode	WR = Inst[20:16] (rt)	WR = Inst[15:11] (rd)
RegWrite	Dec/RegWr	no register write	write WD into register WR
ALUSrc	ALU	op2 = RD2 (register)	op2 = SignExt(Inst[15:0])
ALUcontrol	ALU	4-bit: selects ALU operation (table below)	
MemRead	Memory	no memory read	read Mem[Address]
MemWrite	Memory	no memory write	Mem[Address] ← RD2
MemToReg	RegWrite	WD = ALU result	WD = memory read data
PCSrc	Mem/RegWr	PC ← PC+4	PC ← (PC+4) + SignExt(imm) << 2

! MemToReg mux has its inputs swapped (1 on top = memory). PCSrc = Branch AND isZero.

Main control truth table (input: opcode Inst[31:26]):

opcode	RegDst	ALUSrc	MemToReg	RegWr	MemRd	MemWr	Branch	op1	op0
R-type	000000 (0)	1	0	0	1	0	0	1	0
lw	100011 (2B)	0	1	1	1	0	0	0	0
sw	101011 (2B)	X	1	X	0	0	1	0	0
beq	000100 (4)	X	0	X	0	0	0	0	1

One AND gate per opcode (R-format, lw, sw, beq), then OR where needed: RegDst = ALUop1 = R-format; ALUSrc = lw + sw; MemToReg = MemRead = lw; RegWrite = R-format + lw; MemWrite = sw; Branch = ALUop0 = beq.

ALU control (2-level: opcode → ALUop; ALUop + funct → ALUcontrol):

instr	ALUop	funct	action	ALUctrl	ALUcontrol	Ainv	Binv	Op
lw, sw	00	XXXXXX	add	0010	AND 0000	0	0	00
beq	01	XXXXXX	subtract	0110	OR 0001	0	0	01
add	10	100000	add	0010	add 0010	0	0	10
sub	10	100010	subtract	0110	sub 0110	0	1	10
and	10	100100	AND	0000	slt 0111	0	1	11
or	10	100101	OR	0001	NOR 1100	1	1	00
slt	10	101010	slt	0111				

ALUcontrol3 = 0; ALUcontrol2 = ALUop0 + ALUop1·F1;

ALUcontrol1 = ALUop1 + F2; ALUcontrol0 = ALUop1·(F3 + F0).

(ALUop 11 never occurs, so ALUop1 alone flags R-type (the beq row's MSB becomes X); F5, F4 are 10 for all these, so ignored.)

L11 Datapath basics

Processor = **datapath** (components that process data: arithmetic, logic, memory ops) + **control** (tells datapath, memory, I/O what to do per instruction). Implements subset: add, sub, and, or, addi, andi, ori, slt, lw, sw, beq, bne. andi/ori are **not** truly supported here since the immediate is always *sign-extended*. sll/srl and j not supported (shift = repeated add; $j \approx \text{beq } \$zero, \$zero, L$ ignoring 256MB blocks).

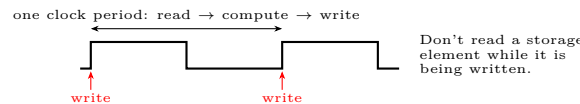
Instruction cycle : Fetch → Decode → Operand fetch → Execute → Result write (each stage's output feeds the next). MIPS: merge Decode + Operand fetch (decoding is simple), split Execute into ALU + Memory → **Fetch, Decode, ALU, Memory, RegWrite**.

	add rd,rs,rt	lw rt,ofst(rs)	beq rs,rt,ofst
Decode	opr1=[rs], opr2=[rt]	opr1=[rs], opr2=ofst	opr1=[rs], opr2=[rt]
ALU	opr1+opr2	MemAddr=opr1+opr2	Taken=(opr1==opr2); Target=(PC+4)+ofst×4
Memory	idle	read Mem[MemAddr]	idle
RegWrite	→ rd	mem data → rt	if Taken, PC=Target

L11 Stage by stage

1 Fetch : read instruction at PC from **instruction memory**; $PC \leftarrow PC+4$ via an adder (next instruction is 4 bytes on, except branch/jump). Output: the instruction.

Clocking: PC is read during the clock period; the new value is written at the next **rising edge** (must be ready before it). Between edges PC is stable.



Elements as functions: IM(addr)=Mem[addr]; Add(A,B)=A+B (combinational); instruction memory is sequential (has state; clock implied).

2 Decode : split instruction Inst[Y:X] = bits X..Y: op [31:26], rs [25:21], rt [20:16], rd [15:11], shamt [10:6], funct [5:0], imm [15:0]. **Register file**: 32×32-bit; reads ≤2 (RR1→RD1, RR2→RD2), writes ≤1 (WR, WD) when **RegWrite**=1.

Problems for I-format (addi \$21,\$22,-50): destination is in rt (not rd) → **RegDst mux**; operand 2 is the immediate ⇒ **sign-extend** 16→32 bits and **ALUSrc mux**. lw needs nothing extra here. beq: outcome and target both needed, solved in ALU stage.

Multiplexer: $n = 2^m$ inputs of equal width, m control bits, output = input_{*i*} when control = *i*. Larger muxes are built from 2-input ones.

3 ALU (execute): arithmetic, shift, logic; address calculation for lw/sw; compare + target for branches. ALU: two 32-bit inputs, 4-bit **ALUcontrol**, outputs result and 1-bit **isZero?** (result==0). beq: subtract, isZero=1 means equal (bne: taken when isZero=0). Branch target needs an extra adder: (PC+4) + (SignExt(imm) << 2) and a **PCSrc mux**.

4 Memory : only lw/sw use it; address from ALU. **Data memory**: Address, Write data (= RD2, for sw), Read data (for lw); **MemRead/MemWrite**, at most one asserted. Others idle; ALU result passes through.

5 RegWrite : route result (ALU or memory, chosen by **MemToReg** mux) to

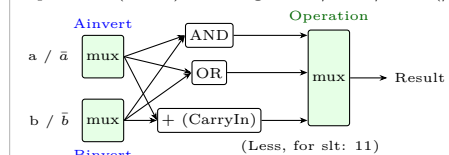
WD; WR number was chosen back in Decode. Stores, branches, jumps write nothing.

L12 Control design

Signals derived from the **opcode** (format) and, for R-type, **funct**. All signals except ALUcontrol come from opcode alone ⇒ most of the controller is opcode-only. Steps: list instruction subset; for each signal see how it depends on op/funct; build truth table; design with gates.

Subset: R-type (op 0; funct add 20, sub 22, and 24, or 25, slt 2A hex), lw 23, sw 2B, beq 4.

1-bit ALU (ALU built one bit at a time): 4 control bits = Ainvert, Binvert, Operation (2 bits) selecting AND / OR / sum (/ less for slt).



Subtract = Binvert + CarryIn 1 (2s comp); NOR = $\bar{a} \cdot \bar{b}$ (Ainvert, Binvert, AND). **Multilevel decoding**: instead of a 12-input function (op+funct), first opcode → 2-bit **ALUop** (00 lw/sw, 01 beq, 10 R-type), then ALUop+funct → ALUcontrol. Simpler design, smaller main controller, potentially faster.

L12 Instruction execution & performance

Every instruction in one clock period: (1) read storage (regs/memory), (2) compute via combinational logic, (3) write storage. Datapath shared by instruction types via MUXes + control signals generated from the encoding.

Single-cycle shortcoming (memory 2ns, ALU/adders 2ns, register file 1ns):

instr	IMem	RegRd	ALU	DMem	RegWr	total
ALU	2	1	2		1	6
lw	2	1	2	2	1	8
sw	2	1	2	2		7
beq	2	1	2			5

Clock must fit the slowest (lw, 8ns) so every instruction takes 8ns.

Fix 1: multicycle: one step (fetch, decode/reg read, ALU, mem, reg write) per clock; shorter cycle, higher frequency; instructions take variable #cycles (COD 5.5, not covered). **Fix 2: pipelining:** steps as above, but different instructions occupy different steps simultaneously (later lecture).

Tracing tips (exam style)

► For a given instruction, walk the 5 stages and read off each mux select: e.g. sw

\$21,-50(\$22): RegDst X, ALUSrc 1, ALUop 00 (add), MemWrite 1, RegWrite 0, MemToReg X, Branch 0.

- Value on a wire: RD1 = [rs]; RD2 = [rt]; SignExt(imm); ALU result = address for lw/sw; branch target = PC+4+imm×4.
- X (don't care) appears when the result of that mux is never written/used (RegDst, MemToReg for sw/beq).
- **bne** needs Branch AND isZero (an extra signal). **addi** would be like lw without memory: RegDst 0, ALUSrc 1, MemToReg 0, RegWrite 1, ALUop 00.